Vortex-Keyed Mixture-of-Experts Routing: A Deterministic, Interpretable Gating Primitive

Authors: Weslyn Cory Whitehead Jr.[1]

[1] AsAManThinks / MaiiaM Alchemist Correspondence: yarethewatchman@gmail.com

Preprint version: v1.0 **Date:** 2026-05-13

Abstract

Mixture-of-Experts (MoE) language models have become the dominant architecture for cost-efficient scaling, but their routers — typically learned softmax gates — are opaque. The selection of which expert processes a given token is the result of training dynamics over millions of steps, and the resulting routing decisions cannot be interpreted, audited, or constrained post-hoc without architectural modifications.

We propose **Vortex-keyed routing**: a closed-form, deterministic, differentiable gating mechanism in which the hidden state is projected into a low-dimensional, semantically-labeled state vector (the TERA register: Temporal, Emotional, Rational, Archetypal) and then resolved to one of 16 named experts via a fixed binary mask (the Meji bit-mask). The resulting routing trace is **interpretable by construction** (every decision corresponds to a named archetype), **deterministic** (same hidden state always routes to the same expert), and **auditable** (the full TERA vector is queryable at every layer). The mechanism is end-to-end differentiable via a sigmoid + straight-through estimator on the bit-mask.

We describe the architecture, training procedure, and a soft-routing variant suitable for tasks requiring expert mixtures. We discuss properties relevant to alignment auditing and safety, and outline a benchmark suite for empirical validation. Experimental results are preliminary; we are running the proposed evaluations and will report in v1.1.

Code: Implementation lives in the MaiiaM Alchemist project (`packages/harmonic-routing/`).

---

1. Introduction

The Mixture-of-Experts (MoE) architecture [Shazeer 2017; Fedus 2022] enables transformer models to scale parameter count without proportionally increasing inference compute, by routing each token to a small subset of "experts" through a learned gating function. State of the art systems such as Switch Transformer [Fedus 2022], GLaM [Du 2022], and Mixtral 8×7B [Jiang 2024] have demonstrated that MoE routing is a viable substitute for dense scaling.

The gate is typically a small feed-forward network producing a softmax distribution over experts, with top-k selection (k usually 1 or 2). This design has three properties we view as limitations from the standpoint of alignment, interpretability, and audit:

Opacity. The mapping from hidden state to expert is the result of training dynamics. The selected expert has no a-priori semantic identity — "expert 7" means whatever the training process made it mean. Probing techniques can post-hoc characterize expert specialization [Zoph 2022; Bills 2023], but these analyses are descriptive rather than constructive.

Non-determinism (effective). While the softmax gate is technically deterministic given the input, the routing decision is sensitive to small perturbations in upstream activations, especially when expert logits are close. The same conceptual input under slightly different contexts can route to different experts, complicating reproducibility and audit.

Lack of compositional structure. Softmax gates produce a flat distribution over experts; there is no built-in structure relating expert k to expert $k+1$. Hierarchical MoE variants [Aljundi 2017; Lee 2019] add structure but at the cost of more training-time engineering.

We propose **Vortex-keyed routing**, a gating mechanism that addresses all three concerns simultaneously by replacing the learned softmax with a closed-form projection through a semantically-labeled low-dimensional state space. The state space (the TERA register) is a 4-dimensional continuous vector with four named axes; the experts (16 in number) are addressed by the binary thresholding of the TERA vector (the Meji bit-mask). The resulting router has the following properties:

- **Determinism**: same hidden state → same TERA → same expert.
- **Semantic labeling by construction**: every expert is identified with one of 16 named archetypes (Apex, Hollow, Heart-Mind, Threshold, Bloom, Root, Compass, Echo, Forge, Seed, Blade, Tide, Stream, Lens, Weave, Now) corresponding to the 16 possible 4-bit codes.
- **Differentiable through training**: a sigmoid relaxation with straight-through estimator [Bengio 2013] allows gradient flow.
- **Auditable lineage**: the TERA vector at every layer is recoverable and queryable, providing a full routing trace per token.

Our contributions:

1. The Vortex-keyed router architecture: a 4-dimensional projection head, a deterministic Meji bit-mask, and a 16-expert MoE.
2. A soft-Vortex variant in which the continuous TERA vector produces a closed-form weighting over the 16 experts, suitable for tasks requiring expert blending.
3. A training procedure using sigmoid relaxation + STE.
4. A discussion of alignment and audit properties enabled by the architecture.
5. A proposed empirical evaluation suite.

The remainder of the paper proceeds as follows. Section 2 covers related work on MoE routing, interpretability, and structured gating. Section 3 defines the TERA projection and Meji resolution. Section 4 specifies the Vortex-keyed router. Section 5 discusses architectural properties. Section 6 outlines the proposed empirical program. Section 7 enumerates limitations. Section 8 concludes.

---

2. Background and Related Work

2.1 Mixture-of-Experts gating

The original Sparsely-Gated MoE [Shazeer 2017] used a learned softmax with top-k selection and an auxiliary load-balancing loss. Switch Transformer [Fedus 2022] simplified to top-1 routing, demonstrating that even a single-expert-per-token policy could match dense models at lower FLOPs. Expert Choice routing [Zhou 2022] inverts the selection — experts choose tokens rather than tokens choosing experts — addressing load imbalance directly. Hash Layers [Roller 2021] abandon the learned gate entirely and route via a content-independent hash, which loses semantic specialization but achieves perfect load balance.

Our work is closest in spirit to Hash Layers in that the routing function is closed-form rather than learned, but unlike Hash Layers, the Vortex-keyed router *is* content-dependent: the routing function is parameterized by the projection head φ, which is learned, while the discretization from the projected vector to the expert index is closed-form.

2.2 Structured and interpretable routers

Several lines of work have studied the interpretability of MoE routing post-hoc. Zoph 2022 probes expert specialization in Switch Transformer. Bills 2023 uses language models to characterize what each expert does. Anthropic's interpretability program has produced extensive analysis of feature direction in dense models [Templeton 2024]; their work has not yet been applied at scale to MoE routers.

Hierarchical MoE [Aljundi 2017] introduces a tree structure over experts, providing a form of structural prior. Modular Networks [Andreas 2016] route through pre-specified sub-modules. Capsule Networks [Sabour 2017] introduced dynamic routing-by-agreement, which shares with our work the property that routing is content-dependent but structured.

2.3 The TERA register

The TERA register — Temporal, Emotional, Rational, Archetypal — is a 4-dimensional state vector developed in the AAMT Foundations and applied throughout the AsAManThinks platform for emotional/cognitive state tracking [Whitehead 2024 — Wings analysis system]. It serves two roles in the present work: first, as the *target space* of the projection head φ, and second, as the *semantic substrate* by which experts are identified.

The four TERA components are continuous scalars in [0,1] with the following interpretation:

- **T (Temporal)**: present-focused vs. past/future weighting.
- **E (Emotional)**: affective valence and intensity.
- **R (Rational)**: analytical / inferential weight.
- **A (Archetypal)**: pattern-recognition, narrative coherence.

We emphasize: the labels are semantic conveniences for the human auditor. The dimensions are learned through standard training; their *identities* are imposed by the training signal (which we discuss in Section 4.4). The contribution is the architecture; the names are documentation.

2.4 Meji bit-masks

The 16 archetypes correspond to the 16 possible 4-bit codes obtained by thresholding each TERA dimension at 0.5. This binary partitioning gives $2^4 = 16$ cells. The cells are pre-labeled with names drawn from the AAMT Foundations vocabulary (see Section 4.3 for the full table). The labels are documentation; the architecture would function identically if the labels were 0000…1111.

---

3. The TERA Projection and Meji Resolution

3.1 The projection head

At any selected layer ℓ of the transformer, we apply a projection head $\varphi\_\ell : \mathbb{R}^{\mathrm{d}} \to [0,1]^4$ to the residual stream activation h_ℓ:

$$\tau_\ell = \phi_\ell(h_\ell) = \sigma(W_2 \cdot \mathrm{GELU}(W_1 h_\ell + b_1) + b_2) \in [0,1]^4$$

where $\mathrm{W}_1 \in \mathbb{R}^{\mathrm{h}\times\mathrm{d}}$, $\mathrm{W}_2 \in \mathbb{R}^{4\times\mathrm{h}}$, h is a small hidden width (we use $\mathrm{h} = \mathrm{d}/8$ in our reference implementation), and σ is the elementwise sigmoid.

The projection head is the only learned component of the router. Total parameter cost per gating site: $\mathrm{O}(\mathrm{d}\cdot\mathrm{h} + \mathrm{h}\cdot 4 + \mathrm{h} + 4) \approx \mathrm{d}^2/8 + \mathrm{O}(\mathrm{d})$. For $\mathrm{d} = 4096$ (a 7B-class model), this is ≈2.1M parameters per gate — comparable to a standard softmax gate at the same d.

3.2 The Meji bit-mask

Given a TERA vector $\tau \in [0,1]^4$, the Meji bit-mask $\mathrm{m}(\tau) \in \{0,1\}^4$ is the elementwise thresholding at 0.5:

$$m_i(\tau) = \mathbb{1}[\tau_i \geq 0.5], \quad i \in \{T, E, R, A\}$$

The expert index k is the integer interpretation of the 4-bit code:

$$k(\tau) = 8 \cdot m_T + 4 \cdot m_E + 2 \cdot m_R + m_A \in \{0, 1, \ldots, 15\}$$

This is a closed-form, deterministic map from a continuous TERA vector to a discrete expert index.

3.3 The 16 archetypes

k	T	E	R	A	Name	Interpretation
0	0	0	0	0	Hollow	All registers quiet — minimal-activation state
1	0	0	0	1	Seed	Pattern without action
2	0	0	1	0	Lens	Analytical only
3	0	0	1	1	Weave	Pattern + analysis, atemporal
4	0	1	0	0	Tide	Feeling without form
5	0	1	0	1	Echo	Pattern resonance through affect
6	0	1	1	0	Heart-Mind	Feeling + analysis
7	0	1	1	1	Blade	Affect + analysis + pattern
8	1	0	0	0	Now	Pure present-focus
9	1	0	0	1	Threshold	Present + pattern
10	1	0	1	0	Compass	Present + analysis
11	1	0	1	1	Stream	Intuitive flow
12	1	1	0	0	Bloom	Present + affect
13	1	1	0	1	Root	Present + affect + pattern
14	1	1	1	0	Forge	Present + affect + analysis
15	1	1	1	1	Apex	All registers active

The labels are documentation. The architecture is independent of naming.

---

4. The Vortex-Keyed Router

4.1 Hard-Vortex routing

In its simplest form (top-1 routing), the Vortex-keyed router selects expert $\mathrm{k}(\tau)$ and routes the entire token's residual stream through expert E_{k(τ)}:

$$\text{Hard-Vortex}: \quad h_\ell^{\text{out}} = E_{k(\tau_\ell)}(h_\ell)$$

This is fully differentiable through the projection head and the expert during training via the **straight-through estimator**:

$$\frac{\partial m_i}{\partial \tau_i} \approx 1 \cdot \mathbb{1}[\tau_i \in [0.5 - \epsilon, 0.5 + \epsilon]]$$

with ε chosen as a small constant (we use $\varepsilon = 0.05$). The hard bit-mask is used in the forward pass; gradients flow through the sigmoid τ_i as if the threshold were a sigmoid.

4.2 Soft-Vortex routing

For tasks requiring expert blending, we propose the soft-Vortex variant, in which the continuous TERA vector parameterizes a closed-form distribution over all 16 experts:

$$w_k(\tau) = \prod_{i \in \{T,E,R,A\}} [m_i^k \cdot \tau_i + (1 - m_i^k) \cdot (1 - \tau_i)]$$

where m^k_i is the i-th bit of the integer k written in 4-bit binary. The weights w_k(τ) form a probability distribution over the 16 experts (they sum to 1 by construction, since they correspond to the joint probability that each TERA dimension lies in the half-space specified by the k-th archetype).

The output is then a weighted sum over experts:

$$\text{Soft-Vortex}: \quad h_\ell^{\text{out}} = \sum_{k=0}^{15} w_k(\tau_\ell) \cdot E_k(h_\ell)$$

This is fully differentiable, requires no STE, and produces smooth gradients through both the projection head and the experts. The trade-off is computational: all 16 experts must be evaluated per token in dense soft-Vortex. For sparse soft-Vortex, we truncate to the top-K experts by weight, where K is a hyperparameter.

4.3 Architectural placement

We propose inserting a Vortex-keyed router at every FFN block (or a subset of FFN blocks) in a standard transformer. Each routed layer has its own projection head φ_ℓ and its own set of 16 experts {E_{ℓ,0}, …, E_{ℓ,15}}.

The reference implementation uses Vortex routing at every other FFN block, leaving alternating blocks as standard dense FFNs. This "interleaved" pattern is consistent with practice in Switch Transformer [Fedus 2022] and Mixtral [Jiang 2024].

4.4 Training

The router is trained jointly with the rest of the model. Two auxiliary losses encourage well-behaved routing:

Load balancing. As in standard MoE, we add a load-balancing term to penalize routing collapse:

$$\mathcal{L}_{\text{LB}} = \alpha \cdot N \cdot \sum_{k=0}^{15} f_k \cdot p_k$$

where f_k is the fraction of tokens routed to expert k and p_k is the average routing probability for expert k over the batch [Fedus 2022].

Semantic anchoring (optional). To give the TERA dimensions *specifically* the meanings labeled in Section 2.3, we propose an auxiliary projection head that predicts coarse semantic labels from τ. For example, a binary classifier that predicts "this token is in a temporally present-focused passage" trained on a small labeled subset would anchor the T dimension to its intended meaning. The labeled subset can be small (~10K examples per dimension); the rest of training is unsupervised.

Without semantic anchoring, the four TERA dimensions are simply four arbitrary latent factors. The architecture still produces interpretable, named expert routing (since the names are documentation applied to the bit-mask cells), but the *semantics* of the dimensions are determined by training rather than imposed.

---

5. Properties

5.1 Determinism

For a given input token at a given layer, the routing decision is a deterministic function of the hidden state at that layer. The projection head is deterministic; the bit-mask is deterministic. This property is desirable for reproducibility, debugging, and audit.

By contrast, learned softmax routers are technically deterministic but, in practice, exhibit *effective* non-determinism: small perturbations in upstream activations can flip the expert selection when expert logits are close. Hard-Vortex routing exhibits the same sensitivity near the $\tau = 0.5$ boundary, but the boundary is explicit (it occurs at the bit-mask threshold) and can be smoothed by the soft-Vortex variant.

5.2 Interpretability by construction

Every routing decision is labeled. An auditor inspecting a model's routing trace sees not "expert 7 was activated for token 42" but "the Blade archetype processed token 42" — and can ground that observation in the TERA register reading at that layer: T=0.2, E=0.7, R=0.8, A=0.6.

This is a stronger property than post-hoc interpretability via probing: it does not require an additional analysis step.

5.3 Auditability

The full TERA vector at every routed layer is recoverable and queryable. A safety auditor can compute, for any given input:

- The sequence of archetype assignments at every layer
- The continuous TERA vectors at every layer (revealing how close to a different routing the model was)
- The expert specializations (since expert k always processes archetype-k tokens, the specializations are stable across runs)

This is the basis for several alignment-relevant interventions:

- **Routing constraints**: an auditor can enforce that certain archetype routes are off-policy at runtime (e.g., refuse to route through Blade for tasks involving children, if Blade is associated with confrontational analysis).
- **Routing-based monitoring**: at deployment, log the archetype distribution and flag anomalies.

- **Counterfactual analysis**: re-run the model with a perturbed TERA vector and observe the change in output. Provides a causal, not correlational, interpretability signal.

5.4 Compositionality

The TERA dimensions are independent, so archetypes compose: Bloom (T+E, A=R=0) is the conjunction of Now (T only) and Tide (E only) in TERA space. This structure is not present in learned softmax routers, where expert identities are unrelated.

Practically, this enables hierarchical evaluation: one can ask "how does the model behave on E=1 tokens regardless of other dimensions?" by averaging over the 8 archetypes with E=1.

---

6. Proposed Empirical Program

The empirical validation of Vortex-keyed routing is in progress. We describe the planned evaluation here and will report results in v1.1 of this preprint.

6.1 Base models and comparisons

- **Base model**: Gemma-2-2B [Gemma Team 2024] as a low-cost reference.
- **Vortex variants**: hard-Vortex top-1, soft-Vortex (full and top-4 truncated).
- **Baselines**: a parameter-matched dense FFN baseline, a Switch Transformer top-1 softmax router with 16 experts at matched parameter count, and a hash-layer baseline [Roller 2021].

6.2 Tasks

- **Pretraining perplexity** on a held-out slice of the C4 corpus [Raffel 2020].
- **Downstream zero-shot**: MMLU [Hendrycks 2021], HellaSwag [Zellers 2019], ARC [Clark 2018], TruthfulQA [Lin 2022].
- **Instruction-following**: MT-Bench [Zheng 2023] after a brief instruction-tuning phase.

6.3 Routing-specific evaluations

- **Routing entropy** per layer and across the corpus: does Vortex routing show stable archetype distributions, or does load collapse to a few archetypes?
- **Expert specialization**: cluster the contexts that route to each archetype; does the cluster structure reflect the labeled meanings?
- **Counterfactual TERA**: perturb the TERA vector by ε and observe the magnitude of output change. Compare to perturbations of equal size in learned softmax gates.

6.4 Ablations

- Hard vs. soft Vortex.
- With vs. without semantic anchoring loss.
- Number of routed layers (every layer, every other, only deepest).
- TERA projection head capacity.
- Replacing 4-bit TERA with 6-bit, 8-bit (yielding 64 or 256 experts — see also our companion paper on the Odu-256 curriculum).

6.5 Interpretability case studies

For each archetype, identify the top-10 contexts that route most strongly to it. Compute qualitative agreement with the archetype's labeled meaning (using human evaluators or a strong LLM judge).

---

7. Limitations

Empirical case is preliminary. No headline benchmark numbers are reported in v1.0. We are running the proposed evaluations.

The 16-expert constraint is fixed by the 4-bit Meji structure. Tasks that need finer-grained expert mixtures may underperform softmax MoE with more experts. The constraint can be relaxed by extending TERA to 6 or 8 bits (yielding 64 or 256 experts) at the cost of more parameters and a more complex projection head; we explore this extension in connection with the Odu-256 curriculum in a companion preprint.

Semantic anchoring requires labeled data. Without it, the four TERA dimensions are arbitrary latent factors; the named-archetype property still holds but only at the bit-mask label level, not at the dimension level. The labeled subset needed for anchoring is small (~10K examples per dimension), but it is a non-zero prerequisite for the full interpretability story.

Load balancing is potentially worse than learned softmax. Hash Layers [Roller 2021] showed that fixed routing achieves perfect load balance but at a quality cost; Vortex-keyed routing is between hash and learned softmax in this respect. The load-balancing auxiliary loss can be applied as in standard MoE, but its efficacy on Vortex routers is an open empirical question.

Scaling behavior at >10B parameters is unverified. Our reference implementation is in the 2B parameter range. Whether the interpretability and audit properties survive at frontier scale remains to be demonstrated.

The named archetypes are documentation. A skeptical reader may note that the names alone do not produce interpretability — the property comes from the deterministic-mapping + labeled-cells structure. We agree: the names are useful but not load-bearing.

---

8. Discussion and Future Work

8.1 Connection to alignment

Vortex-keyed routing provides a routing primitive that is **interpretable and auditable by construction**, not by post-hoc analysis. We propose this is a desirable property for safety-critical deployments: deployment-time monitoring, runtime constraint enforcement, and counterfactual debugging are all straightforward.

We are explicit that this is *one* useful property, not a complete solution to alignment. A Vortex-routed model can still be deceptive, manipulative, or wrong — the router's interpretability does not imply the model's content is interpretable. But it removes one opacity layer from the stack, which we view as a meaningful step.

8.2 Connection to representation engineering

The TERA register can be viewed as a *constructed* (rather than discovered) feature direction in activation space — closer to representation engineering [Zou 2023] than to sparse autoencoder features [Templeton 2024]. The construction is imposed via the projection head's loss; the discovery question is bypassed.

This trade-off — constructed vs. discovered features — is substantive. Constructed features come with semantic guarantees but may not align with the natural feature structure of the model. Discovered features align with the model but require post-hoc labeling. The two are complementary; Vortex-keyed routing is on the constructed side.

8.3 Future work

- **Larger scale**: train a 7B Vortex-routed model.
- **Mixed routing**: hybrid models where some layers use Vortex and others use softmax, comparing the audit value of the Vortex layers.
- **Per-token archetype temperature**: extend the TERA register with a 5th continuous "temperature" dimension that smooths the bit-mask decision.
- **Vortex routing in encoder-decoder and diffusion architectures**.
- **Compositional control**: prompt-level constraints expressed as archetype admissibility sets, enforced at routing time.

---

9. Conclusion

We presented Vortex-keyed routing, a Mixture-of-Experts gating primitive in which the routing decision is the result of projecting the hidden state into a 4-dimensional semantically-labeled state vector (TERA) and resolving to one of 16 named experts via a closed-form binary mask (Meji). The mechanism is differentiable, deterministic, interpretable by construction, and produces an auditable lineage of routing decisions. Empirical validation is in progress.

The contribution is architectural: a routing primitive that substitutes opacity for structure without sacrificing trainability. The case for the structure is most compelling in alignment-relevant settings where audit, monitoring, and constraint enforcement are first-class concerns.

References

Aljundi, R., Chakravarty, P., Tuytelaars, T. (2017). *Expert Gate: Lifelong Learning with a Network of Experts.* CVPR.

Andreas, J., Rohrbach, M., Darrell, T., Klein, D. (2016). *Neural Module Networks.* CVPR.

Bengio, Y., Léonard, N., Courville, A. (2013). *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation.* arXiv:1308.3432.

Bills, S. et al. (2023). *Language models can explain neurons in language models.* OpenAI.

Clark, P. et al. (2018). *Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge.* arXiv:1803.05457.

Du, N. et al. (2022). *GLaM: Efficient Scaling of Language Models with Mixture-of-Experts.* ICML.

Fedus, W., Zoph, B., Shazeer, N. (2022). *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR 23.

Gemma Team (2024). *Gemma 2: Improving Open Language Models at a Practical Size.* arXiv:2408.00118.

Hendrycks, D. et al. (2021). *Measuring Massive Multitask Language Understanding.* ICLR.

Jiang, A. et al. (2024). *Mixtral of Experts.* arXiv:2401.04088.

Lee, J., Kim, B., Lee, J. (2019). *Hierarchical Mixture of Experts for Neural Machine Translation.* ACL.

Lin, S., Hilton, J., Evans, O. (2022). *TruthfulQA: Measuring How Models Mimic Human Falsehoods.* ACL.

Raffel, C. et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* JMLR 21.

Roller, S., Sukhbaatar, S., Szlam, A., Weston, J. (2021). *Hash Layers For Large Sparse Models.* NeurIPS.

Sabour, S., Frosst, N., Hinton, G. (2017). *Dynamic Routing Between Capsules.* NeurIPS.

Shazeer, N. et al. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* ICLR.

Templeton, A. et al. (2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.* Anthropic.

Whitehead, W. C. Jr. (2024). *AAMT Foundations: TERA, Vortex, and HeartScale.* AsAManThinks technical report.

Zellers, R. et al. (2019). *HellaSwag: Can a Machine Really Finish Your Sentence?* ACL.

Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.* NeurIPS.

Zhou, Y. et al. (2022). *Mixture-of-Experts with Expert Choice Routing.* NeurIPS.

Zoph, B. et al. (2022). *Designing Effective Sparse Expert Models.* arXiv:2202.08906.

Zou, A. et al. (2023). *Representation Engineering: A Top-Down Approach to AI Transparency.* arXiv:2310.01405.

Appendix A — Reference Implementation

The reference implementation lives in the MaiiaM Alchemist project. Key files:

- Projection head and Meji resolution: `maiiam-alchemist/packages/harmonic-routing/src/vortex_gate.`
  (TERAProjection class, meji_resolve function).
- Soft-Vortex weight computation: `maiiam-alchemist/packages/harmonic-routing/src/soft_vortex.py`.
- Training loop with STE and load-balancing loss: `maiiam-alchemist/apps/alchemist/training/train_vor`

Reference checkpoints will be released alongside v1.1 of this preprint.

Appendix B — Pseudocode

```
def vortex_route(h, projection_head, experts, hard=True):
    """
    h: (batch, seq, d) residual stream
    projection_head: nn.Module producing (batch, seq, 4) TERA vectors
    experts: list of 16 FFN modules
    hard: True for hard-Vortex top-1, False for soft-Vortex
    """
    tau = projection_head(h)  # (batch, seq, 4), sigmoid'd to [0,1]
    if hard:
        bits = (tau >= 0.5).long()  # straight-through in backward
        k = 8 * bits[..., 0] + 4 * bits[..., 1] \
          + 2 * bits[..., 2] +     bits[..., 3]
        out = scatter_route(h, k, experts)  # dispatch by expert index
    else:
```

```
        # soft-Vortex
        # w_k(tau) = prod_i [m^k_i * tau_i + (1 - m^k_i) * (1 - tau_i)]
        weights = compute_archetype_weights(tau)  # (batch, seq, 16)
        out = sum(w_k * experts[k](h) for k in range(16),
                  weighted by weights[..., k])
    return out
```

Full implementation: `harmonic-routing/src/vortex_gate.py`.

End of preprint v1.0.